North South University

Department of Electrical and Computer Engineering

CSE 311

Database Management System

Project Report

Design and Implementation of a Multi-Vendor E-Commerce Marketplace Database

Submitted by

Name	Student ID	Role
		Database Designer
		Database Administrator
		Application Developer

Submitted to

Course Instructor, CSE 311

Date of Submission: _____

Table of Contents

1. Introduction

1.1 Background

Online retail in Bangladesh has moved away from the single-seller shop model towards marketplaces that host many independent sellers on one platform. Daraz, Chaldal and Pickaboo all follow this pattern: the platform owns the storefront, the payment flow and the delivery integration, while the actual products belong to hundreds of separate vendors. This arrangement is convenient for the customer, who sees one catalogue and pays once, but it places an unusual demand on the database. A single shopping cart may contain items belonging to four different businesses, each of which must be paid separately, must pack and ship its own parcel, and must never be able to see another vendor's orders.

This project designs and implements the database that sits underneath such a marketplace. The work covers the full path from requirement gathering, through conceptual modelling with an entity relationship diagram, to a normalised relational schema running on MariaDB, and finally a desktop application that exercises the database through JDBC.

1.2 Problem Statement

A marketplace database has to reconcile two views of the same purchase that pull in opposite directions. The customer experiences one basket, one payment and one delivery address. Each vendor experiences only its own share of that basket, which it prices, packs and dispatches independently of every other vendor in the same basket. A design that models a purchase as a single order cannot answer the question "what does this vendor owe me and what must it ship?" without scanning and filtering line items every time. A design that models a purchase as several independent orders loses the fact that the customer paid once.

The central design problem addressed by this project is therefore the correct representation of order splitting, together with the supporting structures the marketplace needs around it: a multi-level product catalogue with variants, inventory spread over several warehouses, vendor promotions, delivery tracking, customer reviews and a support ticket desk.

1.3 Objectives

- To analyse the requirements of a multi-vendor e-commerce marketplace and express them as a conceptual entity relationship model with complete cardinality.
- To map that conceptual model to a relational schema, applying the standard mapping rules for generalisation hierarchies, multivalued attributes, composite attributes, derived attributes and many-to-many relationships.
- To normalise the resulting schema to third normal form and to justify every place where data appears to be duplicated.
- To implement the schema in MariaDB with a complete set of integrity constraints, and to populate it with realistic sample data.
- To write a query set that answers genuine business questions using only the SQL features covered in the course.

- To build a desktop application that proves the design works in practice, in particular that a mixed-vendor basket really does split correctly.

1.4 Scope of the Project

The project delivers a working database and a demonstration application. It is deliberately bounded in the following ways.

Included	Excluded
Vendor onboarding, storefronts and product catalogue	Real payment gateway integration
Product variants with size, colour and material options	Live courier tracking APIs
Multi-warehouse inventory with low stock alerts	Search ranking and recommendation engines
Cart, wishlist, checkout and vendor order splitting	Image file storage (only paths are stored)
Payments, shipments and delivery status	Multi-currency and internationalisation
Product reviews and a support ticket desk	Password hashing and session security

Table 1.1 Boundaries of the delivered system

2. Methodology

The project followed the classical four stage database design process: requirement analysis, conceptual design, logical design and physical implementation. Each stage produced an artefact that fed directly into the next.

2.1 Development Environment

Component	Choice	Reason
Database server	MariaDB 10.4 (XAMPP)	Bundled with XAMPP, InnoDB supports the foreign keys the design depends on
Administration	phpMyAdmin	Used in the laboratory sessions, convenient for importing scripts
Storage engine	InnoDB	Enforces referential integrity and supports transactions
Character set	utf8mb4	Stores Bangla text and symbols correctly
Application language	Java SE 8 with Swing	Reuses object oriented programming already studied in the degree
Database connectivity	JDBC, MySQL Connector/J 8.0.33	Standard Java database interface, compatible with MariaDB

Table 2.1 Tools used and the reason for each choice

2.2 Requirement Analysis

Requirements were gathered by examining how existing marketplaces behave and were grouped by the actor they serve.

Vendor requirements

- Register an account, set up a profile and manage storefront settings.
- Create, update and remove products, each of which may have several variants distinguished by size, colour or material.
- See real time stock levels for every variant and receive an alert when stock falls to a threshold.
- Create discounts, either a percentage reduction or a fixed amount, and attach them to selected products.

Buyer requirements

- Register, log in and maintain a set of delivery addresses.
- Browse and search the catalogue and choose a specific variant.
- Save items to a wishlist and add variants to a shopping cart.
- Complete a checkout and follow the delivery status of the resulting parcels.
- Leave a star rating and written review after a purchase.

Platform and support requirements

- Split a single customer basket into one order per vendor for independent fulfilment. This is the defining requirement of the whole system.

- Accept one payment covering the whole basket, which may be settled by a user other than the buyer.
- Accept complaints as tickets, assign them to support staff and record the resolution.
- Take a commission from each vendor, calculated from that vendor's sales.

2.3 Conceptual Design

The requirements were expressed as an entity relationship diagram containing 23 entities and 32 relationships. Four modelling decisions shaped the rest of the project.

Generalisation of the user.

Vendors, buyers and support agents all sign in, all have an email address, a password and a name, but each also carries information that is meaningless for the other two. A vendor has a tax identifier and a commission rate; a buyer has a date of birth and loyalty points; a support agent has an average response time. Rather than repeating the shared attributes three times or padding one table with columns that are null two thirds of the time, the model uses an ISA hierarchy with User as the superclass. The specialisation is total, because every account belongs to one of the three, and disjoint, because no account is both a vendor and a buyer.

Separation of product and variant.

A shirt is one product but it is stocked, priced and sold as separate sizes and colours. Price, barcode and stock therefore belong to the variant, not to the product, while the title, description, brand and category belong to the product. Without this separation the catalogue would have to repeat the description for every colour.

Introduction of the Checkout entity.

This is the answer to the problem stated in section 1.2. A Checkout represents the single act of paying and holds the buyer, the delivery address, the date and the basket total. An Order represents one vendor's share of that checkout and holds the store, the fulfilment status and that vendor's subtotal. One checkout therefore relates to one or many orders. The customer's single payment attaches to the checkout; the packing, dispatch and delivery of goods attaches to the orders. Neither view of the purchase is lost.

Recursive category hierarchy.

Categories nest, so Mobile Phones sits inside Electronics, which sits at the top level. Rather than creating a fixed set of tables for each depth, Category has a recursive relationship with itself: each category optionally names a parent category. The depth of the tree is then unlimited and can be traversed with a self join.

2.4 Mapping the ER Model to Relations

The conceptual model was converted to relations using the standard mapping rules. The table below records how each construct in the diagram became part of the schema.

ER construct	Example	Mapping applied
Strong entity	Product	Becomes a table; the identifier becomes the

		primary key
Generalisation (ISA)	User to Vendor, Buyer, Support	Shared primary key: each subclass table takes user_id as both its primary key and a foreign key to the superclass
Multivalued attribute	phone number of a user	Becomes its own table user_phone with the composite key (user_id, phone), since a repeating column would break first normal form
Multivalued attribute	image of a variant	Becomes the table variant_image
Composite attribute	address made of house, block, street, city	Flattened into separate atomic columns; because three entities reference an address it was also promoted to its own table
Derived attribute	age of a buyer	Not stored. Computed at query time from the stored date of birth
Derived attribute	sub total of an order line	Not stored. Computed as quantity multiplied by unit price
One to many relationship	Store sells Product	Foreign key placed on the many side
Many to many relationship	Variant stored in Warehouse	Becomes the table inventory with a composite primary key and the descriptive attribute quantity
Many to many with attributes	Order references Variant	Becomes order_item, carrying quantity and the unit price snapshot
Weak entity	Review	Identified by the buyer plus the variant, so the primary key is the composite (buyer_id, variant_id)
Recursive relationship	Category parent of Category	Nullable foreign key parent_category_id pointing at the same table

Table 2.2 ER to relational mapping decisions

2.5 Normalisation

Every relation in the schema satisfies third normal form. The reasoning is set out below using the tables where the risk of a violation was highest.

First normal form.

Every attribute is atomic and no relation contains a repeating group. The two attributes that were multivalued in the conceptual model, the user's phone number and the variant's image, were each moved into a separate relation rather than being stored as a comma separated list. Similarly the address, which is composite, was decomposed into house, block, street, city, postal code and country so that a query can filter on city alone.

Second normal form.

No non-key attribute depends on part of a composite key. The relation most at risk is order_item, whose candidate key is (order_id, variant_id). Its attributes quantity and unit_price depend on the whole key: quantity is meaningless without knowing both which order and which variant, and unit_price records what that variant cost in that specific order. The product title was deliberately not placed here, because it would depend on variant_id alone and so would break second normal form.

Third normal form.

No non-key attribute depends transitively on the primary key. An earlier draft of the schema carried a city column on the store relation. Since city is determined by the address and the address is determined by the store, this is a transitive dependency, and it was removed by referencing the address relation instead. The same reasoning removed the vendor's business name from the store relation: the business name is determined by vendor_id, so it is stored once on vendor and reached through the foreign key.

A justified exception.

The attribute order_item.unit_price duplicates product_variant.price at the moment of purchase. This is not a normalisation error. The two values answer different questions: product_variant.price is the price the shop is asking today, while order_item.unit_price is the price the customer actually agreed to pay on the day of the order. If a vendor raises a price next month, every historical invoice must continue to show the old figure. Storing the snapshot is the standard treatment of a time dependent fact, and removing it would destroy the accounting record.

2.6 Physical Implementation

The logical design was written as a DDL script producing 27 tables. Integrity is enforced by the database rather than left to the application wherever the database is capable of doing so.

Constraint type	Count	Purpose in this schema
PRIMARY KEY	27	Entity integrity; every row is uniquely identifiable
FOREIGN KEY	41	Referential integrity between related tables
UNIQUE	9	Business keys such as email, barcode, tax identifier, promotion code
CHECK	17	Domain rules such as star ratings between 1 and 5, non negative stock, an end date not earlier than a start date
NOT NULL	many	Mandatory participation taken from the ER diagram
Secondary index	5	Supports the search and reporting queries

Table 2.3 Integrity constraints implemented in the schema

Deletion behaviour was chosen per relationship rather than applied uniformly. Cascading deletion is used where a child row has no meaning without its parent, for example a product variant without its product. Restricted deletion protects accounting history, so a buyer who has placed a checkout cannot be removed while that record exists. Setting the foreign key to null is used where the reference is optional, for example a ticket whose support agent leaves the company.

2.7 Application Layer

A Java Swing application connects through JDBC and provides one dashboard per role. The login screen resolves the ISA hierarchy by probing the three subclass tables after the credentials match, and opens the corresponding window. The interface uses a navigation rail,

summary tiles and coloured status pills so that the state of an order is readable at a glance, and it includes a drawn diagram of the vendor split described in section 3.3.

All database access uses prepared statements with bound parameters. Besides preventing SQL injection, this keeps the value formatting out of the string, which matters for dates and decimals.

The checkout routine is the only place where the application performs a multi-statement update, and it runs inside a single transaction. It reads the cart joined to the store that owns each item, groups the lines by store, writes one checkout row and one order row per store, deducts stock, records the payment and empties the cart. If any line fails its stock check, the whole transaction is rolled back and no partial basket is left behind.

3. Use Case Analysis

3.1 Actors

Actor	Description	Represented in the database by
Buyer	A registered customer who browses, purchases and reviews products	buyer (subclass of user)
Vendor	An independent seller operating one or more storefronts	vendor (subclass of user)
Support Agent	Platform staff who resolve customer complaints	support (subclass of user)
Courier	External delivery company carrying parcels to the buyer	shipment_company

Table 3.1 Actors in the system

3.2 Use Case Summary

ID	Use Case	Actor	Main tables involved
UC-01	Register and log in	All	user, vendor, buyer, support
UC-02	Manage storefront	Vendor	store
UC-03	List a product with variants	Vendor	product, product_variant, variant_option, variant_image
UC-04	Monitor stock and receive low stock alerts	Vendor	inventory, warehouse, product_variant
UC-05	Create a promotion	Vendor	promotion, promotion_variant
UC-06	Browse and search the catalogue	Buyer	vw_product_catalog
UC-07	Add a variant to the cart	Buyer	cart_item
UC-08	Save a variant to the wishlist	Buyer	wishlist_item
UC-09	Check out a mixed vendor basket	Buyer	checkout, customer_order, order_item, payment, inventory
UC-10	Update order status	Vendor	customer_order
UC-11	Dispatch and track a shipment	Vendor, Courier	shipment, shipment_item, shipment_company
UC-12	Review a purchased product	Buyer	review
UC-13	Raise a support ticket	Buyer	ticket
UC-14	Resolve a support ticket	Support Agent	ticket
UC-15	View vendor sales and commission	Vendor	vw_vendor_sales

Table 3.2 Use cases supported by the database

3.3 Primary Scenario: Multi-Vendor Checkout

This scenario is the reason the Checkout entity exists and is the use case the demonstration walks through.

Field	Detail
Use case	UC-09 Check out a mixed vendor basket

Primary actor	Buyer
Goal	Pay once for a basket whose items belong to different vendors, and have each vendor receive only its own order
Precondition	The buyer is logged in, the cart is not empty and at least one delivery address is saved
Postcondition	One checkout row, one order per vendor, matching order lines, one payment, reduced stock and an empty cart

Main flow

1. The buyer opens the cart. The system lists each line with its store, quantity and sub total, the sub total being computed rather than read from a column.
2. The buyer selects a delivery address and a payment method, then confirms.
3. The system reads the cart joined to the product and store tables, so that every line now carries the identity of the vendor who must fulfil it.
4. The system checks the requested quantity of every line against the total stock held across all warehouses.
5. The system writes one checkout row recording the buyer, the address and the date.
6. The system groups the cart lines by store. For each group it writes one order row carrying that store, then writes the order lines beneath it, copying the current variant price into unit_price as a snapshot.
7. The system totals the orders, writes that figure back to the checkout, and records a single payment against the checkout.
8. The system deducts the sold quantities from the warehouses, taking from the warehouse holding the most of that variant first.
9. The system empties the cart and commits the transaction.

Alternative flow: insufficient stock.

If step 4 finds any line whose requested quantity exceeds the stock on hand, the transaction is rolled back in full. No checkout, order, payment or stock movement is written and the buyer's cart is left untouched, so the basket can be corrected and submitted again.

Worked example.

The sample data contains 355 baskets that crossed vendor boundaries. The table below lists those that fanned out to three separate sellers: one payment by one customer, three independent orders to pack and dispatch.

Checkout	Buyer	Vendor Orders	Stores	Total
8	Mamun Sarkar	3	Khan Bazar / Saha Mart / Yasmin Gallery	9567.48
11	Taslima Sarwar	3	Karim Fashion / Mehedi Gallery / Papia Mart	30917.07
13	Shamim Kundu	3	Ayesha Hub / Chakma Collection / Sheikh Electronics	132152.54
15	Rehana Parvin	3	Chakma Mart / Mahmud Bazar / Rasel Store	293024.41
21	Ruma Akter	3	Chowdhury Fashion / Mahmud Bazar / Saiful Gallery	99736.93

27	Dilruba Ghosh	3	Alamgir Enterprise / Ayesha Hub / Lamia Electronics	16438.33
28	Trisha Dey	3	Ayesha Hub / Molla House / Umme Fashion	132708.89
69	Chanchal Sultana	3	Ayesha Hub / Sarwar Collection / Shirin Emporium	19822.30

Table 3.3 Baskets that fanned out to three separate sellers

3.4 Supporting Scenarios

Low stock alert (UC-04).

Each variant carries its own restock threshold, because a laptop selling a few units a month and a phone charger selling hundreds cannot share one figure. The alert sums the stock held in every warehouse for a variant and reports it when the total has fallen to or below that variant's own threshold.

Store	Product	Variant	On Hand	Threshold
Sheikh Traders	Leather Sandal	Size L - Charcoal	0	10
Biswas Electronics	Shin Guard Pair	Sky Blue	0	10
Umme Fashion	Vostro 3420	Graphite - 3 Year Warranty	0	20
Mushfiqur Traders	Turmeric Powder 200g	Pack of 1	0	10
Molla Fashion	Digital Sports Watch	White	0	8
Sarwar Collection	Rajshahi Silk Saree	Free Size - Sky Blue	0	5

Table 3.4 Variants currently needing a restock

Category navigation (UC-06).

The recursive category relationship is read with a self join, in which the category table is opened twice under two different aliases so that a row can be matched against its own parent.

Sub Category	Parent Category
Baby Care	Baby & Kids
Baby Food	Baby & Kids
Diapers	Baby & Kids
Toys	Baby & Kids
Fragrance	Beauty & Health
Haircare	Beauty & Health

Table 3.5 Category hierarchy resolved by a self join

4. Entity Relationship Diagram

The diagram below shows the conceptual design as it was finally implemented. Entities drawn with a double outline are associative entities, which exist to resolve a many-to-many relationship that carries attributes of its own. Cardinality uses crow's foot notation, where the symbol nearest an entity states how many occurrences of that entity take part in the relationship.

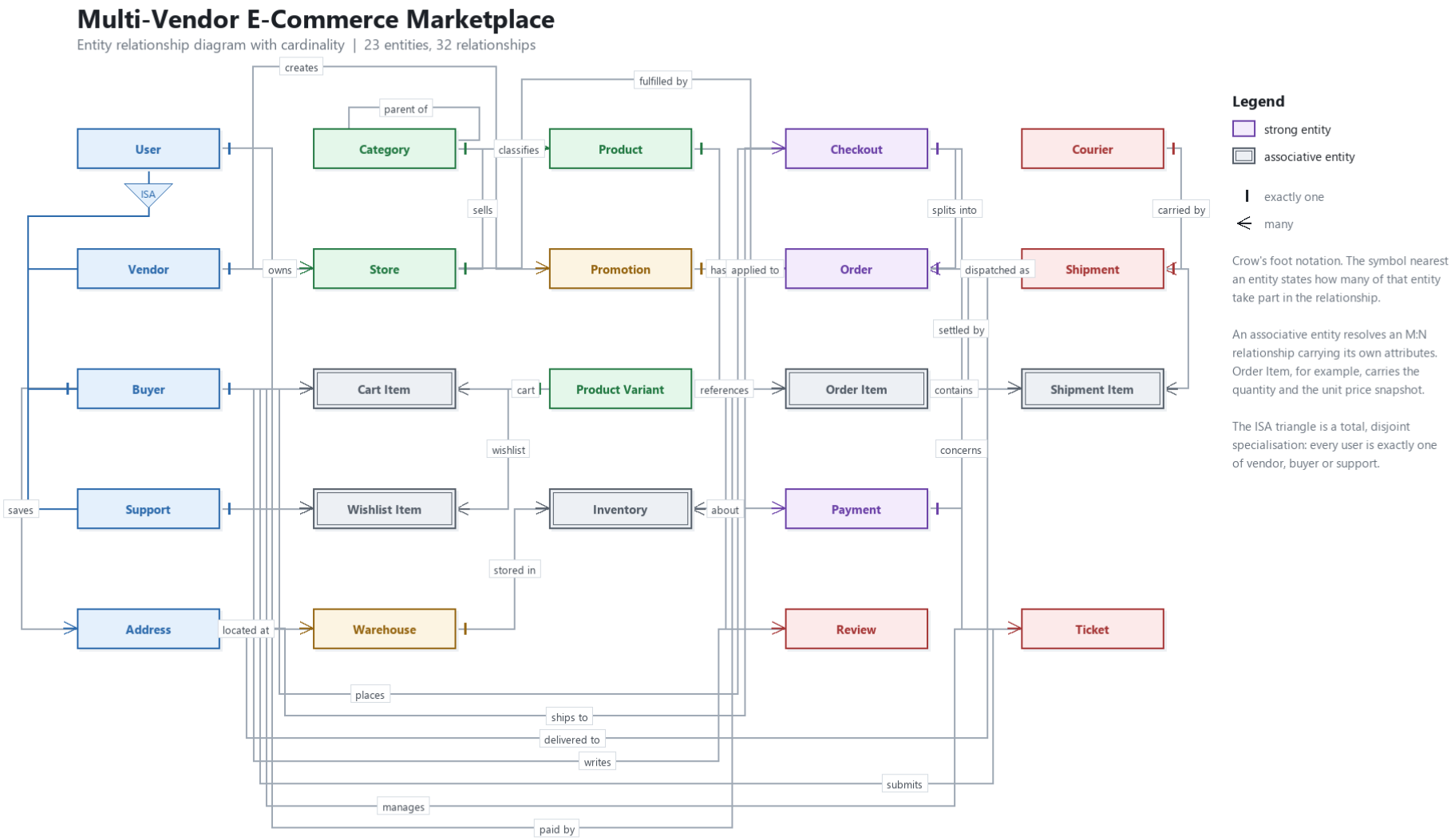

Figure 4.1 Entity relationship diagram with cardinality

Relationship catalogue

Every relationship is listed below with its cardinality and the participation of each side. Total participation means an occurrence of that entity cannot exist without the relationship.

Relationship	Cardinality	Participation	Meaning
Vendor owns Store	1 : N	Vendor partial, Store total	A vendor may run several storefronts; every storefront has an owner
Store sells Product	1 : N	Store partial, Product total	Every product is listed by exactly one storefront
Category classifies Product	1 : N	Both partial	A product may be left uncategorised
Category parent of Category	1 : N	Both partial	Recursive; a top level category has no parent
Product has Variant	1 : N	Both total	A product is sold only through its variants
Variant stored in Warehouse	M : N	Both partial	Resolved by Inventory, which carries the quantity
Buyer saves Address	1 : N	Both partial	A buyer may store several addresses
Warehouse located at Address	N : 1	Warehouse total	Every warehouse has one address
Vendor creates Promotion	1 : N	Promotion total	Every promotion belongs to a vendor
Promotion applies to Variant	M : N	Both partial	One offer may cover many variants
Buyer carts Variant	M : N	Both partial	Resolved by Cart Item, which carries the quantity
Buyer wishlists Variant	M : N	Both partial	Resolved by Wishlist Item
Buyer places Checkout	1 : N	Checkout total	Every checkout belongs to one buyer
Checkout ships to Address	N : 1	Checkout total	One delivery address per basket
Checkout splits into Order	1 : N	Both total	The defining relationship; one basket becomes one order per vendor
Checkout settled by Payment	1 : 1	Payment total	One payment covers the whole basket
User pays Payment	N : 1	Payment total	The payer need not be the buyer
Order fulfilled by Store	N : 1	Order total	Each split order belongs to one storefront
Order references Variant	M : N	Order total	Resolved by Order Item, carrying quantity and the unit price snapshot
Order dispatched as Shipment	1 : N	Shipment total	An order may go out in several parcels
Shipment carried by Courier	N : 1	Shipment total	One courier per parcel
Shipment contains Order Item	M : N	Both partial	Resolved by Shipment Item, which allows a partial dispatch
Shipment delivered to Address	N : 1	Shipment total	Destination of the parcel
Buyer writes Review	1 : N	Review total	A review must have an author
Review about Variant	N : 1	Review total	A review targets one variant
Buyer submits Ticket	1 : N	Ticket total	Every ticket has a complainant
Support manages Ticket	1 : N	Both partial	A new ticket may be unassigned
Ticket concerns Order	N : 1	Both partial	A complaint need not be about an order

Table 4.1 Cardinality and participation of every relationship

5. Schema Diagram

The relational schema produced by the mapping rules of section 2.4 contains 27 tables. Primary keys are underlined, foreign keys are marked FK, and arrows run from a foreign key to the primary key it references.

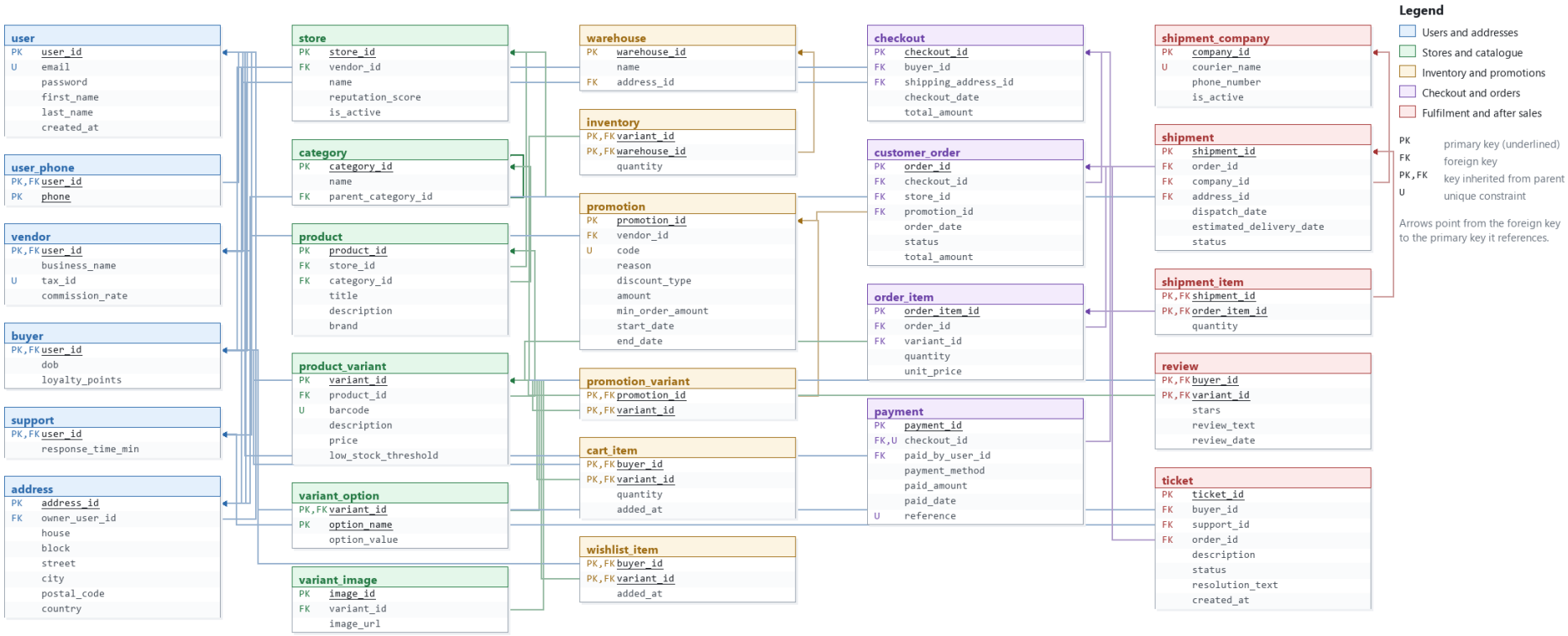

Figure 5.1 Relational schema diagram

Tables and populated volume

The sample data set contains 23,704 rows spread over the 27 tables. It is modelled on the Bangladeshi retail market: local names, districts and areas, local brands and products, local courier companies and mobile financial services. Every table is populated, so no query in the demonstration returns an empty result by accident.

Table	Rows	Table	Rows
address	622	shipment	1043
buyer	372	shipment_company	10
cart_item	377	shipment_item	2095
category	58	store	66
checkout	732	support	22
customer_order	1161	ticket	150
inventory	3361	user	432
order_item	2336	user_phone	558
payment	688	variant_image	3350
product	655	variant_option	2172
product_variant	1672	vendor	38
promotion	41	warehouse	8
promotion_variant	404	wishlist_item	497
review	784		

Table 5.1 Tables in the schema and rows loaded into each

Views

Six views sit on top of the base tables. They hide the larger joins behind a single name, allow the application to query one object instead of five, and keep sensitive columns such as the password out of every screen.

View	Purpose
vw_product_catalog	Browse screen: variant, product, category, storefront and total stock in one row, restricted to active stores
vw_vendor_sales	Revenue, units sold, platform commission and vendor earning per vendor, excluding cancelled orders
vw_order_summary	One row per split vendor order with buyer, store, line count and delivery state
vw_low_stock	Variants whose total stock has reached their own restock threshold
vw_product_rating	Average rating per variant, keeping variants that have no reviews yet
vw_buyer_profile	Buyer summary including the derived age and lifetime spend, excluding the password column

Table 5.2 Views defined on the schema

6. Implementation and Results

6.1 Coverage of Course Material

The query set was written to exercise every SQL feature covered in the laboratory sessions. One deliberate extension goes beyond Weeks 1-7: a trigger and a stored procedure from Week 9, added to close a limitation noted in Section 7 and listed as the last row of Table 6.1.

Week	Topic	Where it appears in the project
1 and 2	CREATE TABLE, INSERT, SELECT, aliases, arithmetic, concatenation, DISTINCT	schema.sql and seed.sql; queries A1 to A6
3	WHERE, BETWEEN, IN, LIKE, IS NULL, AND, OR, NOT, ORDER BY	Queries B1 to B11; catalogue search in the application
4	Equijoin, join with ON, self join, three way join, LEFT, RIGHT and FULL OUTER JOIN	Queries C1 to C10; the category self join and the seven table order query
5	COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING, nested group functions	Queries D1 to D11; vendor revenue and low stock reports
6	Single row subqueries, multi row subqueries with IN, ANY and ALL, group functions inside subqueries	Queries E1 to E11; the order splitting proof
7	CREATE VIEW	views.sql defines six views; queries F1 to F6 read from them
8 and 9	CREATE TRIGGER, CREATE PROCEDURE	db/routines.sql; the procedure is called from the vendor Overview screen (VendorFrame.java)

Table 6.1 Mapping of laboratory topics onto the delivered project

6.2 Sample Results

Vendor revenue and commission.

This report joins vendor, store, order and order line, excludes cancelled orders, groups by vendor and computes the platform commission from the vendor's own commission rate. The commission is calculated rather than stored, so changing a rate takes effect immediately.

Vendor	Orders	Units	Gross Revenue	Commission	Vendor Earning
Karim International	50	171	5916983.17	487559.41	5429423.76
Hossain International	32	111	2448760.89	217205.09	2231555.80
Sultana Traders	33	90	1569951.50	92941.13	1477010.37
Trisha Brothers	33	108	1366793.22	69979.81	1296813.41
Saiful International	39	119	1124033.95	96666.92	1027367.03
Sheikh Trading	58	191	837101.35	51398.02	785703.33
Rezaul Enterprise	24	83	798348.17	38320.71	760027.46
Sikder Traders	18	50	738009.40	69963.29	668046.11
Shirin Agencies	45	148	724883.89	26820.70	698063.19
Ayesha Trading	49	160	612826.02	54602.80	558223.22

Table 6.2 Revenue and commission, ten highest earning vendors

Product ratings.

Ratings are aggregated across all variants of a product, so a customer sees one figure for the product rather than a separate score for each colour.

Product	Brand	Reviews	Average Stars
Steel Water Bottle 1L	Marcel	3	5.00
Dishwash Liquid 1L	Harpic	3	5.00
UHT Milk 1L	Pran	3	5.00
Leather Sandal	Adidas	3	5.00
Kajal Waterproof	Maybelline	5	5.00
Dishwash Liquid 1L	Vim	4	5.00
Boys Cotton Panjabi Set	Yellow	6	4.83
Matte Liquid Lipstick	Maybelline	4	4.75

Table 6.3 Highest rated products with at least three reviews

6.3 Representative Queries

Order splitting, expressed as a single query. Any basket appearing here was paid for once but fulfilled by more than one vendor.

```
SELECT  ck.checkout_id,
        CONCAT(u.first_name,' ',u.last_name) AS buyer,
        COUNT(o.order_id)                    AS vendor_orders,
        ck.total_amount
FROM        checkout       ck
JOIN        `user`         u ON u.user_id     = ck.buyer_id
JOIN        customer_order o ON o.checkout_id = ck.checkout_id
GROUP BY ck.checkout_id, u.first_name, u.last_name, ck.total_amount
HAVING  COUNT(o.order_id) > 1;
```

Self join reading the recursive category hierarchy.

```
SELECT  child.name  AS sub_category,
        parent.name AS parent_category
FROM    category child
JOIN    category parent ON child.parent_category_id = parent.category_id;
```

Multi-row subquery finding variants that have never been ordered.

```
SELECT  pv.variant_id, pv.barcode, pv.description, pv.price
FROM    product_variant pv
WHERE   pv.variant_id NOT IN (SELECT variant_id FROM order_item);
```

Group function inside a subquery, comparing each store against the average store.

```
SELECT  s.name, SUM(oi.quantity * oi.unit_price) AS revenue
FROM        store          s
JOIN        customer_order o  ON o.store_id  = s.store_id
JOIN        order_item     oi ON oi.order_id = o.order_id
GROUP BY s.name
HAVING  SUM(oi.quantity * oi.unit_price) > (
            SELECT AVG(store_total) FROM (
                SELECT SUM(oi2.quantity * oi2.unit_price) AS store_total
                FROM   customer_order o2
                JOIN   order_item     oi2 ON oi2.order_id = o2.order_id
```

```
                GROUP BY o2.store_id
            ) AS t
        );
```

6.4 Testing

The database was verified by rebuilding it from the scripts and running a set of consistency checks, each written so that a correct database returns no rows.

Check	Result
Checkout total equals the sum of its vendor order totals	No mismatches
Every order line belongs to a variant sold by that order's store	No violations
Every promotion applied to an order belongs to that order's vendor	No violations
No shipment line exceeds the quantity actually ordered	No violations
Every review corresponds to a real purchase by that buyer	No violations
A rating outside one to five is rejected	Rejected by CHECK constraint
An order line pointing at a non existent variant is rejected	Rejected by FOREIGN KEY constraint
Two orders for the same store within one checkout are rejected	Rejected by UNIQUE constraint
A checkout that oversells stock leaves no partial rows behind	Transaction rolled back, no rows written

Table 6.4 Verification checks and their outcome

7. Limitations

The following limitations are known and are stated deliberately rather than left for the reader to discover.

7.1 Rules That Cannot Be Enforced by the Schema

Some business rules are beyond what declarative constraints can express. The course syllabus does cover triggers and stored procedures (Week 9); the stock-oversell rule below now uses one, closing the ordinary-use case, though not the fully concurrent one. The remaining rules are enforced in the Java layer instead, which means they can be bypassed by writing directly to the database through phpMyAdmin.

Rule	Why a constraint cannot express it	Current enforcement
A review may only be written after a confirmed purchase	Requires checking rows in three other tables at insert time	Checked in the application; a trigger would be the proper solution
Stock must never be oversold under concurrent checkouts	The CHECK constraint stops a negative value but cannot serialise two simultaneous buyers	A single transaction with an application stock check, backed by a BEFORE INSERT trigger (trg_order_item_stock_guard) that rejects the insert outright if stock is insufficient. Row level locking (SELECT ... FOR UPDATE) would still be needed for true concurrency between two simultaneous buyers.
An order's total must equal the sum of its lines	Cannot be expressed as a CHECK across two tables	Computed and written by the application inside the same transaction
A promotion may only apply to variants sold by the vendor who created it	Requires a join to validate	Validated in the application

Table 7.1 Rules enforced outside the schema

7.2 Limitations of the Database Platform

- MariaDB does not implement FULL OUTER JOIN, although the syntax is part of the SQL standard and was covered in the laboratory. The query set demonstrates the accepted workaround, a UNION of a LEFT and a RIGHT outer join.
- Passwords are stored as plain text so that the sample accounts can be used during the demonstration. A production system would store a salted hash and would never keep the original value.
- The schema stores image paths as text rather than the image files themselves, which is normal practice, but it means the database cannot guarantee that a referenced file actually exists.

7.3 Functional Limitations

- Payment is recorded but not processed. There is no integration with bKash, Nagad or any card gateway, so the payment row represents an assumed successful transaction. Refunds are not modelled at all.
- Courier tracking is stored as a status column that a vendor updates manually. No courier API is called.
- Returns and exchanges are not modelled. A cancelled order simply changes status and does not restore stock.
- The application does not provide screens for every table. Warehouses, couriers and categories are managed directly in phpMyAdmin.
- Search is a simple LIKE pattern match on the product title and brand. There is no relevance ranking, no spelling tolerance and no full text index.

7.4 Scale

The sample data set of 23,704 rows is large enough to make every query return a meaningful result and to make the reporting views do real work, but it is still far short of the volumes a live marketplace would carry. Index choice was reasoned about rather than measured, and query plans were not profiled. At production volumes the reporting views, which aggregate across order lines every time they are read, would need to be replaced by summary tables refreshed on a schedule.

8. Conclusion

This project set out to design and implement the database behind a multi-vendor e-commerce marketplace, and to do so using only the techniques covered in the course. The finished system consists of a normalised schema of 27 tables with 41 foreign keys and 17 check constraints, six views, a query set spanning every topic from the laboratory sessions, a populated sample data set, and a Java desktop application that drives the whole thing through JDBC.

The most instructive part of the work was the order splitting requirement. The first version of the model treated a purchase as a single order, which read naturally from the customer's point of view but made the vendor's view expensive and awkward: there was no way to attach a fulfilment status or a shipment to one vendor's share of a basket. Introducing the Checkout entity above the Order resolved the tension, because it separated the act of paying, which happens once, from the act of fulfilling, which happens once per vendor. This is a good illustration of a general lesson: when two actors disagree about what constitutes one transaction, the disagreement usually means an entity is missing from the model.

The mapping stage reinforced how much of good schema design comes from a small set of rules applied consistently. Recognising that the phone number was multivalued, that age was derived, and that the relationship between variant and warehouse carried an attribute of its own, each led directly to a structural decision that would have been difficult to reverse later. Equally important was learning when not to apply a rule: the unit price stored on an order line looks like redundancy but is in fact a historical record, and normalising it away would have destroyed the ability to reproduce a past invoice.

The limitations recorded in section 7 point to the natural next steps. One of them has already been taken: a trigger now backstops the stock-oversell rule directly in the database. The remaining Java-enforced rules are natural candidates for the same treatment. Row level locking would make concurrent checkout fully safe. A returns and refunds subsystem would complete the order lifecycle. None of these change the core design; each extends it, which suggests the foundation is sound.

Appendix A: File Manifest

File	Contents
db/schema.sql	Data definition for all 27 tables with every constraint
db/seed.sql	Sample data, 23,704 rows covering every table
db/views.sql	The six views
db/queries.sql	Query set organised by laboratory week
app/src/marketplace/	Java source: login, buyer, vendor and support dashboards, reporting console, checkout service
app/build.bat, app/run.bat	Compile and launch scripts
docs/er-diagram.png	Figure 4.1
docs/schema-diagram.png	Figure 5.1
README.md	Setup instructions

Order of execution

```
mysql -u root < db/schema.sql
mysql -u root < db/seed.sql
mysql -u root < db/views.sql
mysql -u root marketplace_db < db/queries.sql
```

Appendix B: Contribution of Group Members

Member	Role	Contribution
	Database Designer	Requirement analysis, entity relationship diagram, cardinality and participation, sections 1 and 3 of this report
	Database Administrator	ER to relational mapping, DDL and constraints, sample data, view and query design, normalisation analysis, sections 2, 5 and 6
	Application Developer	Java Swing application, JDBC layer, transactional checkout with vendor splitting, testing, sections 4, 7 and 8